

WEEK-6 Assignment

Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

- **Driver:** The main program that creates the Spark session, converts user code into tasks, schedules jobs, and collects results.
- **Cluster Manager:** Allocates resources (CPU and memory) across the cluster and launches executors. Examples include Standalone, YARN, Kubernetes, and Mesos.
- **Executor:** Worker processes that execute tasks, store data in memory or disk, and return results to the driver.

Q2. How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Spark does not execute transformations immediately. Instead, it records them in a **Directed Acyclic Graph (DAG)**. Execution begins only when an **action** is called. This allows Spark to optimize the execution plan, reduce unnecessary computations, minimize data movement, and combine operations efficiently.

Q3. Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
df = spark.read \  
  .option("header", "true") \  
  .option("inferSchema", "true") \  
  .csv("data/source.csv")
```

Q4. What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?

CSV	Parquet
Row-based storage	Columnar storage
Larger file size	Compressed, smaller file size
No schema support	Stores schema information

CSV	Parquet
Slower for analytics	Faster for analytical queries

Parquet reads only the required columns, reducing disk I/O and memory usage, making it much faster for big data analytics.

Q5. Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

```
df.filter(df.category == "Electronics") \
    .select("product_id", "price")
```

Q6. Write the code to revise a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

```
from pyspark.sql.functions import col

df = df.withColumnRenamed("old_name", "new_name") \
    .withColumn("price", col("price").cast("double"))
```

Q7. How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?

Spark keeps track of all transformations in a **Lineage Graph (DAG)**. If a worker node fails and data is lost, Spark recomputes only the lost partitions using the lineage instead of rerunning the entire application. This provides efficient fault tolerance without replicating all data.

Q8. Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

```
df_orders.filter(
    (df_orders.status == "Completed") &
    (df_orders.amount > 1000)
)
```

Q9. Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.

Predicate Pushdown allows Spark to apply filter conditions while reading the Parquet file. Only the required rows and columns are loaded into memory, reducing disk reads, memory consumption, and improving query performance.

Q10. Write a code snippet to add a new column final_price which is the base_price multiplied by 1.18 (18% tax).

```
from pyspark.sql.functions import col
```

```
df = df.withColumn(  
    "final_price",  
    col("base_price") * 1.18  
)
```

Q11. What is the difference between Transformations and Actions? Provide two examples of each.

Transformations	Actions
Create a new DataFrame/RDD Trigger execution and return results	
Lazy evaluation	Immediate execution

Examples of Transformations:

- filter()
- select()

Examples of Actions:

- show()
- collect()

Q12. Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

```
df = spark.read.parquet("path/to/input")
```

```
df.filter(df.user_id.isNotNull()) \
  .write \
  .option("header", "true") \
  .csv("path/to/output")
```

Q13. In Spark Architecture, what is the difference between Client Mode and Cluster Mode?

Client Mode	Cluster Mode
Driver runs on the user's machine	Driver runs inside the cluster
Suitable for development/testing	Suitable for production
If client disconnects, application may stop	Continues even if client disconnects

Q14. Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
df.filter(
  (df.region == "North") |
  (df.priority == "High")
)
```

Q15. When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

- .show(5) displays only the first **5 rows**, making it fast and memory-efficient.
- .collect() retrieves **all rows** from the cluster to the driver, which can consume huge memory and may cause an **OutOfMemoryError** on large datasets.
- Therefore, .show(5) is much safer for exploring very large datasets.